토스플레이스 Node.js Developer 지원 포트폴리오

상담 한 건이 들어와 사건이 되기까지, 그 운영 흐름을 시스템으로 만들어 왔어요

법무법인 법승에서 9개월 동안 사내 시스템 5종을 만들고 운영하고 있어요. 사람이 손으로 하던 일을 찾아 시스템으로 옮기고, 옮긴 뒤에는 잘 돌고 있는지 다시 재요.

유입
상담 접수와 고객 등록
CRM

배정
지사와 담당 변호사
CRM

예약
상담 일정, 구글 캘린더 반영
CRM

안내
문자 종류 자동 판정, 예약 발송
CRM

수임 후
계약, 문서, 결재, 정산
사건관리 앱, 2인 팀

다음
단말기 한 대의 발주부터 재출고까지
토스플레이스

진솔 010-9914-3907 mikely329@gmail.com github.com/sorrybro2

한눈에 보기

9개월 동안 다섯 개 시스템을 만들고 운영해요

2026.01부터

사내 CRM

전국 10개 지사 직원

혼자 개발

Express, Flutter, PostgreSQL

API 66개
테이블 17개

2026.05부터

홈페이지 V2와 어드민 CMS

방문자와 운영 직원

혼자 개발

Next.js, NestJS, Prisma

API 146개
어드민 37화면

2026.06부터

사건관리 앱

변호사와 직원, 지사 베타

2인 팀

React, FastAPI, PostgreSQL

내 커밋 비중
23%

2026.08부터

사내 포털

전 직원

혼자 개발

Next.js 16, Workspace API

Route Handler 22개
별도 DB 없음

연동 서비스

전화 수신 알림, 시트 동기화

혼자 개발

Cloud Run 서비스, Job

서비스 3개

시스템 사이를 잇는 일

CRM → 사건관리 앱
FDW와 API로 당사자 정보를 함께 써요

레거시 홈페이지 → 홈페이지 V2
매일 HTTPS 동기화 배치가 돌아요

사내 포털 → CRM
재직자 명단과 권한을 요청마다 읽어요

사내 포털 → 구글 어드민
계정과 지사별 내선을 직접 써요

규모는 2026.10.12 측정이에요. 합계 API 423개, 테이블 85개, 내 커밋 1,873건이며 사건관리 앱(2인 팀) 수치가 들어 있어요.

겉으로 보인 문제에서 멈추지 않고, 고친 뒤에는 다시 재요

쪼개 보기

불편을 들으면 로그와 데이터로 먼저 나눠요.

"로그인이 느리다"를 나눠 보니 로그인은 1초 안쪽, 느린 건 그다음 화면이었어요.

사례 3에서 자세히

본질 찾기

증상 대신 그 증상을 만든 구조를 찾아요.

동기화가 멈춘 것보다, 멈춘 걸 2주 동안 아무도 몰랐던 구조가 문제였어요.

사례 2, 5에서 자세히

경계 정하기

값과 책임을 어느 시스템이 가질지 먼저 정해요.

바깥에서 DB로 들어오는 길은 백엔드 API 하나로 모았어요.

사례 2, 3에서 자세히

다시 재기

고친 뒤에는 효과를 원인별로 다시 재요.

7.3초에서 4.48초, 1.50초. 원인 하나를 걷어낼 때마다 쟀어요.

사례 3에서 자세히

그리고 왜 이렇게 만들었는지는 문서와 이슈에 남겨요.

사례 1 사내 CRM

업무를 멈추지 않고, 구글시트를 CRM으로 옮겼어요

문제
전 직원이 당사자, 상담, 문자 기록을 구글시트에 손으로 적고 있었어요.

본질
새 시스템을 만드는 동안에도 상담 접수가 멈추지 않는 게 더 중요했어요.
개발과 운영은 혼자 맡았어요.

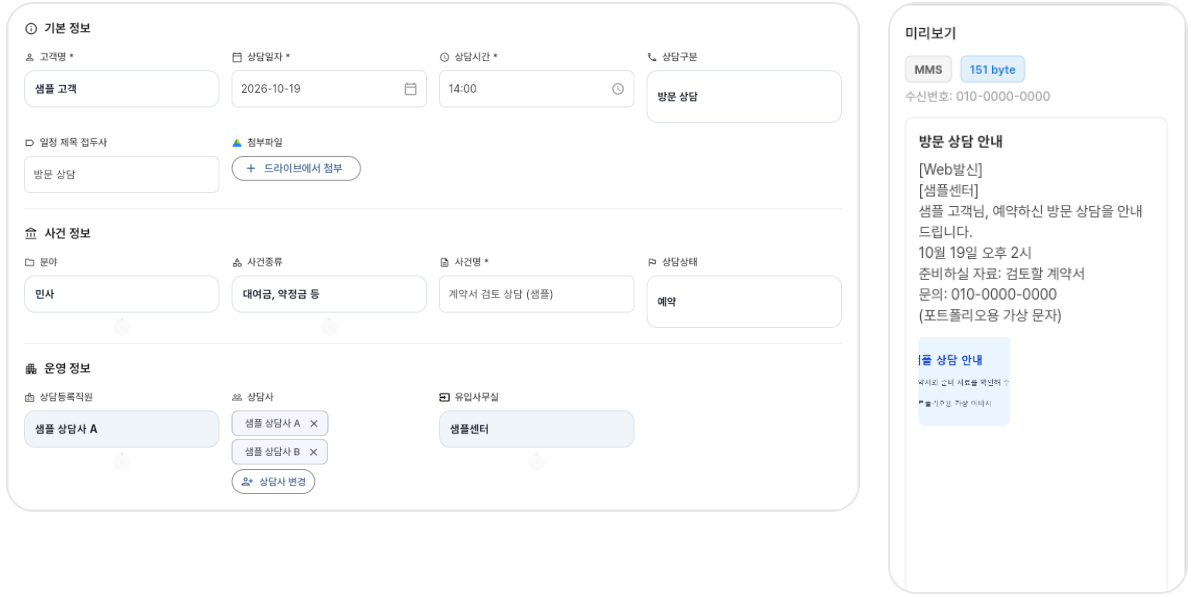
선택
약 9.8만 건을 먼저 옮기고, Apps Script 트리거와 웹훅으로 시트와 DB를 양방향 동기화했어요. 과도기 동안 두 방식이 함께 돌았어요.

결과
2026년 5월 정식 오픈 뒤 전국 10개 지사가 CRM으로 일하고 있어요.

약 9.8만 건
처음 옮긴 기록

2026.05
정식 오픈

10개 지사
지금 쓰는 곳



왼쪽은 상담 등록, 오른쪽은 예약 문자 발송 미리보기예요. 글자 수가 아니라 바이트 수로 문자 종류를 정해요. 합성 데이터로 실행한 실제 화면

사례 1 사내 CRM

서버가 늘어나도 작업은 한 번만, 접수 건은 한 사람만 처리해요

스케줄러 중복 실행



Cloud Run이 인스턴스를 늘리자 동기화 작업이 인스턴스마다 돌았어요. 저장소를 하나 더 두는 대신, 이미 쓰던 MySQL의 `GET_LOCK`으로 한 대만 실행하게 했어요. 혼자 운영하는 서비스라 관리할 인프라를 늘리지 않는 쪽을 골랐어요.

같은 접수 건 동시 처리

처리 시작
직원 A가 처리를 시작하면 점유를 기록해요.

동시 접근
직원 B가 같은 건을 열면 처리 중으로 보이고 수정이 막혀요.

10분 무조작
10분 동안 조작이 없으면 점유가 풀려 다른 직원이 이어서 처리해요.

그 밖에도 페이지당 최대 480개이던 서브쿼리를 JOIN과 복합 인덱스로 바꿨고, 문자는 글자 수가 아닌 바이트 수로 SMS, LMS, MMS를 자동 판정해요.

사례 2 홈페이지 V2, 데이터 이관과 동기화

731만 행을 옮긴 뒤에도, 전환 날까지 다시 맞췄어요

문제
레거시 SQL Server의 41개 테이블을 PostgreSQL로 옮겨야 했어요. 레거시는 전환 날까지 운영 중이라 데이터가 계속 바뀌었어요.

본질
한 번 옮기고 끝나는 일이 아니었어요. 기존 행을 건드리지 않는 증분 방식으로 7주를 보낸 뒤 재 보니, 레거시에서 바뀐 값들이 V2에 쌓이지 않고 있었어요.

선택
테이블 성격마다 맞추는 방법을 나누고, 수정까지 반영하는 전량 업서트와 덮어쓸 값을 미리 보여 주는 점검을 더했어요.

결과
41개 테이블 7,310,476행을 테이블별 행 수 대조로 검증해 옮겼어요. 컷오버 런북에는 마지막 동기화를 전량 업서트로 하도록 넣었어요.

테이블 성격별로 다시 맞추는 방법

새 행만 추가	마지막 번호보다 큰 행만 넣어요	게시글
없는 번호만 보충	V2가 만든 행은 두고 빠진 번호만 채워요	상담
전체 교체	지우고 다시 넣어요. 한 트랜잭션 안에서	작은 표 18개
전량 업서트	번호 기준으로 넣거나 고쳐요	전환 직전

증분 방식으로 7주 뒤 재 보니

526건
레거시에서 숨긴 글이 V2에는 계속 노출

51건
담당 변호사 배정이 반영되지 않음

24건
V2 테스트 접수가 번호를 먼저 차지해 빠진 실제 상담

이관 때 날짜는 한국시간에 UTC 표시만 붙은 채 옮겨졌어요. 이 하나가 이후 역이관과 동기화에서 9시간 보정으로 계속 돌아왔어요.

사례 2 홈페이지 V2, 레거시 연동

DB로 들어오는 길을 API 하나로 모아, 멈춰도 모르던 동기화를 고쳤어요

문제

레거시 홈페이지의 글과 상담을 옮기는 배치가 2주 동안 스케줄러에는 성공으로 떠 있었지만 실제로는 멈춰 있었어요.

본질

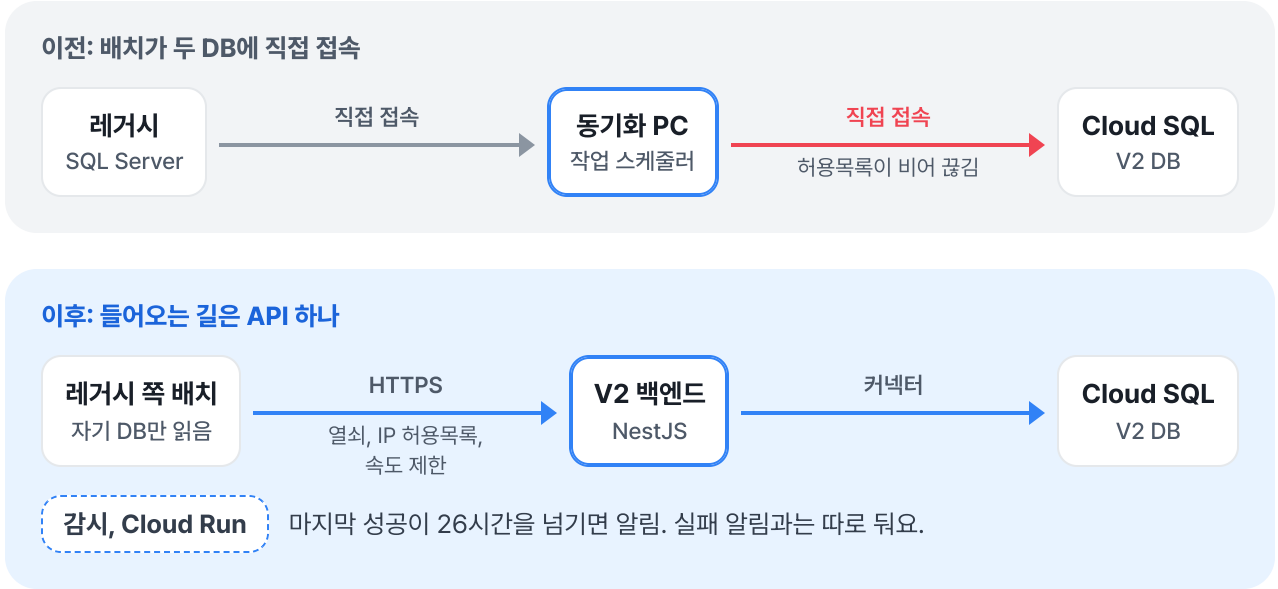
포트 차단으로 적었던 원인을 다시 재 보니 끊긴 곳은 클라우드 DB 허용목록이었어요. 배치가 두 DB에 직접 붙어 끊길 곳이 두 군데였고, 마지막 명령이 echo라 실패해도 종료코드가 0이었어요.

선택

레거시 쪽이 자기 DB를 읽어 V2 백엔드의 HTTPS API로 밀어 넣게 바꿨어요. 고정 IP(연 약 60만 원)와 레거시 서버에 런타임을 얹는 안은 버렸어요.

결과

인바운드 개방과 허용목록 의존이 사라졌고 추가 비용은 0원이에요. 손으로 돌리던 후처리도 같은 API 경로로 옮겼어요.



사례 2 홈페이지 V2, 레거시 연동

빠진 데이터는 겹친 것과 숨은 것을 나눠서 되살렸어요

305건

번호가 겹친 상담

두 사이트가 같은 번호를 서로 다른 사람에게 발급하고 있었어요. 번호로 맞췄다면 42명의 접수가 로그도 없이 사라졌을 거예요. 접수 고유키로 대조하고, 넣을 때 새 번호를 발급해요. 원래 번호는 따로 보관해 재실행하면 0건이 들어가요.

61건

멈춘 사이 밀린 게시물

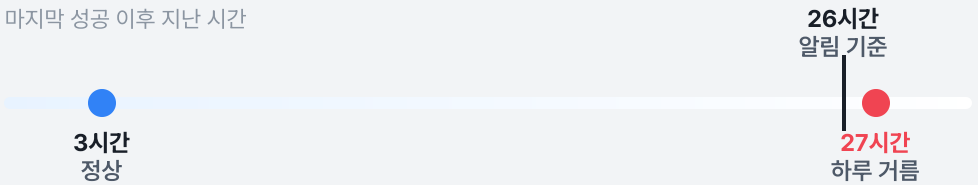
레거시에 있고 V2에 없는 번호만 골라 넣어요. V2 어드민에서도 글을 쓰기 때문에, 최대 번호 다음부터 붙이는 방식으로는 새 글을 놓쳐요.

38건

숨어 있던 예약 글

레거시는 예약 글을 비공개로 넣었다가 당일 공개로 바꾸는데, 배치가 이미 옮긴 글을 다시 보지 않았어요. 매 회차 최근 60일 글의 공개 여부와 작성일을 맞추게 했어요.

매일 09:15에 돌고 12:15에 감시해요. 정상이면 마지막 성공에서 3시간, 하루를 거르면 27시간이 지나 있어요. 그래서 기준을 26시간으로 잡았고, 감시는 대상과 다른 기계(Cloud Run)에 뒀어요. 멈춘 쪽은 스스로 멈췄다고 말할 수 없으니까요.



크롬 확장 프로그램에 있던 구성원 관리를 포털로 옮겼어요



조직도에서 사람 칸을 누르면 옆에서 바로 고쳐요. 바뀐 것은 모았다가 한 번에 저장해요.



퇴사는 날짜와 계정 정지 시각을 정해 예약해요



예약한 퇴사는 명부에 바로 보여요

경계부터 정했어요

CRM에 API를 두는 안은 CRM이 쓰지 않는 값이라 접고, 포털이 구글 어드민에 직접 쓰게 했어요. 이름과 내선은 CRM, 조직도 배치는 포털이 원본이에요.

권한은 요청마다 판정해요

비어 있던 편집자 명단이 "검사 안 함"으로 읽혀 전 직원이 조직도를 저장할 수 있었어요. 이제 CRM으로 판정하고, 못 읽으면 막아요.

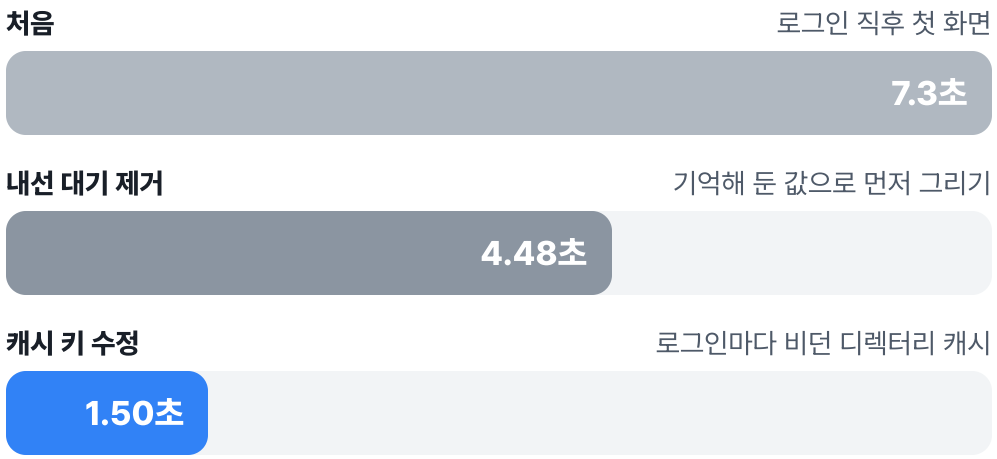
고치면 바로 반영돼요

구성원을 고치면 수신 알림 번호표 Job을 바로 다시 돌려, 최대 30분 옛 값으로 가던 알림을 없앴어요.

합성 데이터로 실행한 실제 화면

사례 3 사내 포털

느린 원인을 하나씩 걷어내며 7.3초를 1.50초로 줄였어요



로그인 직후 첫 화면 응답 시간. 운영 요청 로그로 원인 하나를 고칠 때마다 다시 쟀어요.

문제

"로그인이 느리다"는 의견을 로그로 나눠 보니 로그인 자체는 1초 안쪽이었어요. 느린 건 로그인 직후 첫 화면이었어요.

본질

화면이 지사별 내선 약 20명분(한 사람 1.5~4.9초)을 다 받을 때까지 기다렸어요. 남은 시간은 캐시였어요. Next.js fetch 캐시는 요청 헤더까지 키에 넣어서, 새 access token이 섞인 디렉터리 캐시가 로그인마다 비었어요.

선택

내선은 기억해 둔 값으로 먼저 그리고 뒤에서 갱신해요. 디렉터리는 누가 읽어도 같은 값이라 인스턴스 메모리에 하나만 둬요. 전날 CRM 응답 캐시에서 찾은 것과 같은 함정이었어요.

결과

콜드스타트는 최소 인스턴스 1대로 막고, 화면이 더는 기다리지 않게 된 연결 서비스는 0대로 되돌렸어요.

사례 4 홈페이지 V2, 사례 요약과 FAQ

사례 3,358건에 요약과 FAQ를 붙이는 일을 검사까지 자동으로 돌게 했어요

고르기
공개 사례를 조회수 순으로 30건씩

만들기
Gemini가 요약 3줄과 FAQ 초안을 써요

검사하기
원문에 없는 숫자, 비율, 보장, 최고, 전관, 무료 상담 표현을 잡아요

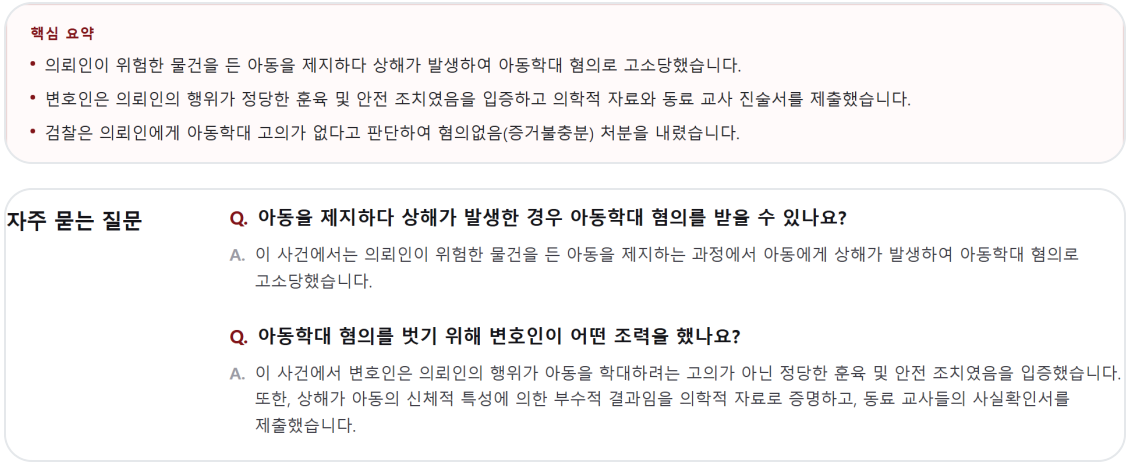
공개하기
통과하면 공개, 걸리면 비공개. 생성 5분, 승인 10분 주기

3,038건
운영 투입 당일 공개

303건
검사에 걸려 비공개

0건
실패

- 본질은 생성보다 검증이었어요. 법률 광고라 결과를 보장하거나 과장하는 문장이 나가면 안 됐어요.
- 생성 결과는 레거시 동기화가 닿는 원본과 분리된 테이블에 둬서, FAQ를 못 읽어도 글은 그대로 나가요.
- 광고 규정을 다시 읽고, AI 표기 없이 실제로 검토한 변호사만 표시하게 설계를 바꿨어요.
- 글이 바뀌면 내용 지문으로 감지해 FAQ를 다시 만들고 검색엔진에 알려요. 첫 통지는 7,227건이었어요.



실제 공개 화면, bslawwin.co.kr

사례 5 운영

장애 세 번에서 원칙 세 개를 만들었어요

사건관리 앱, 2인 팀

API 27분, 9분 중단

소스 배포가 루트 Dockerfile 대신 프런트 이미지를 골라, API가 전부 HTML을 돌려줬어요. 배포는 성공으로 떠 있었어요.

원칙

이미지를 명시해 빌드하고, 배포 뒤 응답 본문과 Content-Type까지 확인해요. 이후 새 시스템에도 같은 절차를 써요.

홈페이지 V2

전면 500, 6분 39초

DB 커넥터가 잠깐 끊기자 NestJS 기동 훅이 재시도 없이 예외를 던져 인스턴스가 계속 죽었어요. DB 자체는 정상이었어요.

원칙

기동 때 0.5초에서 8초 간격으로 5번 재시도하고, 끝내 실패해도 죽지 않고 나중에 연결해요.

레거시 연동

2주 동안 침묵

마지막 명령이 echo라 종료코드가 늘 0이었고, 그중 일주일은 아예 실행되지 않았어요.

원칙

실패 알림과 따로, 돌지 않은 것을 다른 기계에서 감시해요.

세 번 모두 "배포됐다", "성공했다"는 표시가 "제대로 돌고 있다"를 보장하지 않았어요. 그래서 권한 판정도 원본을 못 읽으면 막고, 실패는 삼키지 않고 로그에 남겨요.

함께 만든 프로젝트 사건관리 앱, 2인 팀

엔드포인트 50곳을 고치지 않고 감사로그와 실시간 갱신을 붙였어요

변경 수집
세션 훅이 변경을 모으고, 업무 데이터와 감사로그를 같은 트랜잭션에 저장해요.

커밋 후 발행
커밋이 끝난 뒤에만 PostgreSQL NOTIFY를 보내요.

접속자별 전송
LISTEN 브로커가 접속자별 큐로 나눠 SSE로 보내요.

화면 반영
브라우저는 관련 TanStack Query 캐시만 다시 불러요.

620건, 조기 종료 0건
운영 24시간 SSE 연결

8초에서 0.05초
원격 조인을 배치 조회로

1,695ms에서 109ms
타입 불일치 인덱스 복구

- 2인 팀에서 상담, 문서, 결재, 정산 도메인과 배포를 맡았어요. 제 커밋은 저장소 전체의 23%예요.
- Redis Pub/Sub 대신 이미 쓰던 PostgreSQL `LISTEN/NOTIFY`를 골라 추가 인프라 비용이 없어요.
- 토스플레이스가 쓰는 MikroORM도 Unit of Work와 flush 이벤트로 같은 구조를 가져요.

왜 이렇게 만들었는지를 사람과 AI가 같이 읽게 남겨요

142건

일일 업무 보고

커밋이 있는 날마다 git 로그와 이슈를 근거로 남겨요. AI 에이전트가 같은 규칙으로 초안을 쓰도록 작성 규칙 문서(AGENTS.md)를 만들어 운영해요.

40건

설계와 운영 문서

홈페이지 V2 33건, CRM 7건이에요. 이관 계획, 컷오버 런북, 운영 설정, 직원용 사용자 매뉴얼까지 저장소에 함께 둬요.

커밋과 이슈

커밋은 한글 제목과 본문을 필수로 쓰고, 원인 분석과 결정 근거는 이슈에 남겨요. 보고서에는 커밋 해시와 이슈 번호를 근거로 붙여요.

일일 업무 보고 (2026. 09. 18)

> 2주째 조용히 멈춰 있던 레거시→V2 동기화를 되살려 게시물 61건, 상담 305건을 복구하고 매일 도는 구조로 바꿨다.

10초 핵심 요약

- 2주 침묵의 원인은 한 곳이었다: 끊긴 건 Cloud SQL 허용목록뿐이었다
- 방향을 뒤집어 방화벽 의존을 없앴다: 당겨오는 대신 레거시가 HTTPS로 밀어 넣는다
- 알림을 두 종류로: 실패 즉시 + 26시간 침묵. 실패가 아니라 미실행이면 실패 알림만으로는 또 놓친다

이하 상세 업무 내역, 커밋 해시와 이슈 번호 병기

AI 에이전트 작성 규칙, AGENTS.md 일부

1. 보고 기준은 그날 git 로그 전체. 세션에서 한 일만 적지 않는다
2. 커밋 해시와 이슈 번호를 근거로 병기한다
3. 커밋이 없어도 이슈와 코멘트로 남은 운영 작업은 넣는다
4. 수치는 지표 파일에서만 인용하고, 없으면 직접 세서 먼저 기록한다

> **코드는 AI도 잘 읽지만, 왜 지금 이 모양인지는 기록에만 남아요.**

공고의 기술 옆에 제가 푼 문제를 놓아 봤어요

공고에 적힌 기술과 일	제가 해 본 것
TypeScript, NestJS	홈페이지 V2 백엔드를 NestJS로 혼자 만들었어요. API 146개와 배치, 스케줄러를 운영해요.
MySQL	CRM을 MySQL로 운영하며 `GET_LOCK`으로 중복 실행을 막았어요. 이후 PostgreSQL로 옮겼어요.
MikroORM 다른 도구로 푼 경험	SQLAlchemy 세션 훅으로 Unit of Work 구조를 깊게 다뤘어요. 감사로그를 업무 데이터와 같은 트랜잭션에 넣었어요.
Redis가 맡는 일 다른 도구로 푼 경험	분산 락은 `GET_LOCK`, 실시간 전달은 `LISTEN/NOTIFY`, 캐시는 인스턴스 메모리로 풀었어요. 서버가 많아지면 이 자리를 Redis가 맡게 돼요.
외부 시스템 연동	Google Workspace API, 공공데이터포털 API, GA4 Data API, NCP SENS, 레거시 SQL Server와 연동했어요.
Internal Product	React 업무 화면, 어드민 CMS 37화면, 사내 포털을 직접 만들었어요.
AI	Gemini로 요약 파이프라인과 상담봇을 만들었고, 업무 기록은 Claude Code와 Codex가 읽는 규칙으로 정리해요.

다음에는 단말기 한 대의 흐름에서 사람이 손으로 하던 일을 찾고 싶어요

단말기 한 대가 발주부터 재출고까지 지나는 길에서, 반복되는 수작업을 찾아 시스템으로 옮기고 옮긴 뒤에 얼마나 줄었는지 재는 일을 하고 싶어요.



토스플레이스 Node.js 챕터 소개 글에 나온 단말기 한 대의 라이프사이클이에요.

진솔 010-9914-3907 mikely329@gmail.com github.com/sorrybro2

사내 화면은 실제 소스를 합성 데이터로 실행해 찍었어요. 회사 코드와 고객 데이터는 공개하지 않아요.